5G Standalone Architecture Implementation Using Open5GS and UERANSIM

Detailed Laboratory Report (Experiments 5–10)

Abstract

This report documents the successful implementation and evaluation of a complete 5G Standalone (SA) architecture on Ubuntu using Open5GS and UERANSIM. The work covered core network deployment, Radio Access Network simulation, User Equipment registration, network slicing, Mobile Edge Computing (MEC), and an integrated smart healthcare use case. The experiments demonstrated practical deployment of 5G core systems, low-latency architectures, and service differentiation.

Introduction

5G Standalone architecture uses a dedicated 5G Core and supports advanced capabilities including network slicing, ultra-reliable low-latency communication (URLLC), and Mobile Edge Computing. This lab implemented Open5GS as the core, UERANSIM as gNB and UE simulator, WebUI for subscriber management, and practical edge computing demonstrations.

Experiment 5: Open5GS Core Setup

Objective: Deploy and configure Open5GS core network.

Procedure:
- Installed Open5GS and MongoDB
- Started WebUI from ~/open5gs/webui using npm
- Logged into localhost:9999
- Added subscriber IMSI 999700000000001 with APN 'internet'

Result:
- Open5GS core operational
- WebUI operational
- Subscriber successfully configured

Experiment 6: gNB Integration

Objective: Connect simulated gNB to Open5GS core.

Procedure:
- Configured open5gs-gnb.yaml
- Started gNB using nr-gnb

Result:
- NG Setup successful
- gNB connected to AMF

Experiment 7: UE Registration and PDU Session

Objective: Register UE and establish PDU session.

Procedure:
- Configured open5gs-ue.yaml with subscriber credentials
- Resolved initial SQN mismatch
- Successfully completed registration

Result:
- Registration Accept
- PDU Session Establishment Accept
- uesimtun0 created with IP 10.45.0.2

Experiment 8: Network Slicing

Objective: Demonstrate multiple network slices.

Procedure:
- Modified amf.yaml for SST 1 and SST 2
- Added dual slices in subscriber profile
- Tested UE with SST 1 and SST 2 separately

Result:
- Multi-slice registration demonstrated
- Logical network slicing validated

Experiment 9: Mobile Edge Computing

Objective: Compare edge and cloud latency.

Procedure:
- Started local edge server
- Pinged localhost and 8.8.8.8

Results:
- Edge RTT ≈ 0.030 ms
- Cloud RTT ≈ 123 ms

Conclusion:
- Edge processing drastically reduces latency

Experiment 10: End-to-End Smart Healthcare Use Case

Objective: Integrate 5G Core, slicing, UE, and MEC.

Scenario:
- Smart healthcare / patient monitoring
- URLLC via SST 2
- Edge node via localhost

Results:
- Edge RTT ≈ 0.029 ms
- Cloud RTT ≈ 109 ms

Conclusion:
- 5G SA + URLLC + MEC supports mission-critical smart healthcare

Overall Comparative Summary

Experiment 5: Core deployment successful
Experiment 6: gNB integration successful
Experiment 7: UE registration successful
Experiment 8: Network slicing successful
Experiment 9: MEC latency validated
Experiment 10: End-to-end smart healthcare demonstrated

Conclusion

This project successfully implemented a practical 5G Standalone architecture using Open5GS and UERANSIM. It validated key 5G technologies including subscriber management, UE connectivity, network slicing, and Mobile Edge Computing. The integrated smart healthcare scenario demonstrated the real-world potential of 5G for intelligent, low-latency, mission-critical applications.